

Title: PolyAlphabetic Cryptograph C codes

Name: Joshua Il B Dajao, Vonzelle Cuizon

Section: 1030-130 PM M-F

Date Performed: 7/19/25

Instructor: Sir Godwin Monserate

Overview:

This project implements a **hybrid encryption model** combining both classical and modern cryptographic techniques to achieve enhanced data confidentiality. The algorithm named **V-MVRTH**, applies six layers of transformation, including symmetric and asymmetric encryption, substitution, permutation, and binary-level obfuscation.

The document includes flowcharts for both **encryption** and **decryption** processes.

Step-by-Step Encryption Process

1. Plaintext Input

The encryption process begins with the user-provided plaintext. This is the original, human-readable message that is to be securely encoded.

2. Key Generation

- An RSA key pair is generated: a public key (e, n) for encryption and a private key (d, n) for decryption.
- A Vernam key (one-time pad) is also generated or derived from a secure random source to be used in XOR-based encryption.

3. Monoalphabetic Cipher (Fixed Shift Substitution)

A simple Caesar cipher is applied as a first layer of confusion. Each character in the plaintext is shifted by a fixed number of positions in the alphabet. This step provides an initial transformation to obscure the original characters.

4. Vigenère Cipher (Polyalphabetic Substitution)

A Vigenère cipher is used to apply a keyword-based shifting of characters.

Unlike the monoalphabetic cipher, the shift amount varies based on the repeating keyword, adding complexity and resistance to frequency analysis.

5. **Character Swapping (Positional Permutation)**

The intermediate result is further obfuscated by a series of deterministic character swaps or rotations. This step increases diffusion by rearranging character positions, breaking direct sequential patterns.

6. **Vernam Cipher (XOR Operation)**

Each character from the transformed message is converted to its ASCII representation and XORed with the corresponding character or byte from the Vernam key. This binary-level symmetric encryption adds randomness and non-linearity.

7. **Transpositional Cipher (Block-Level Rearrangement)**

The XOR output undergoes another layer of permutation by reversing or shuffling blocks of characters. This reinforces the diffusion property of the ciphertext and further distorts character positions.

8. **RSA Encryption (Asymmetric Encryption)**

The final output from the previous stages is encrypted using RSA. Each numeric representation of a character or block is encrypted using the RSA public key formula:

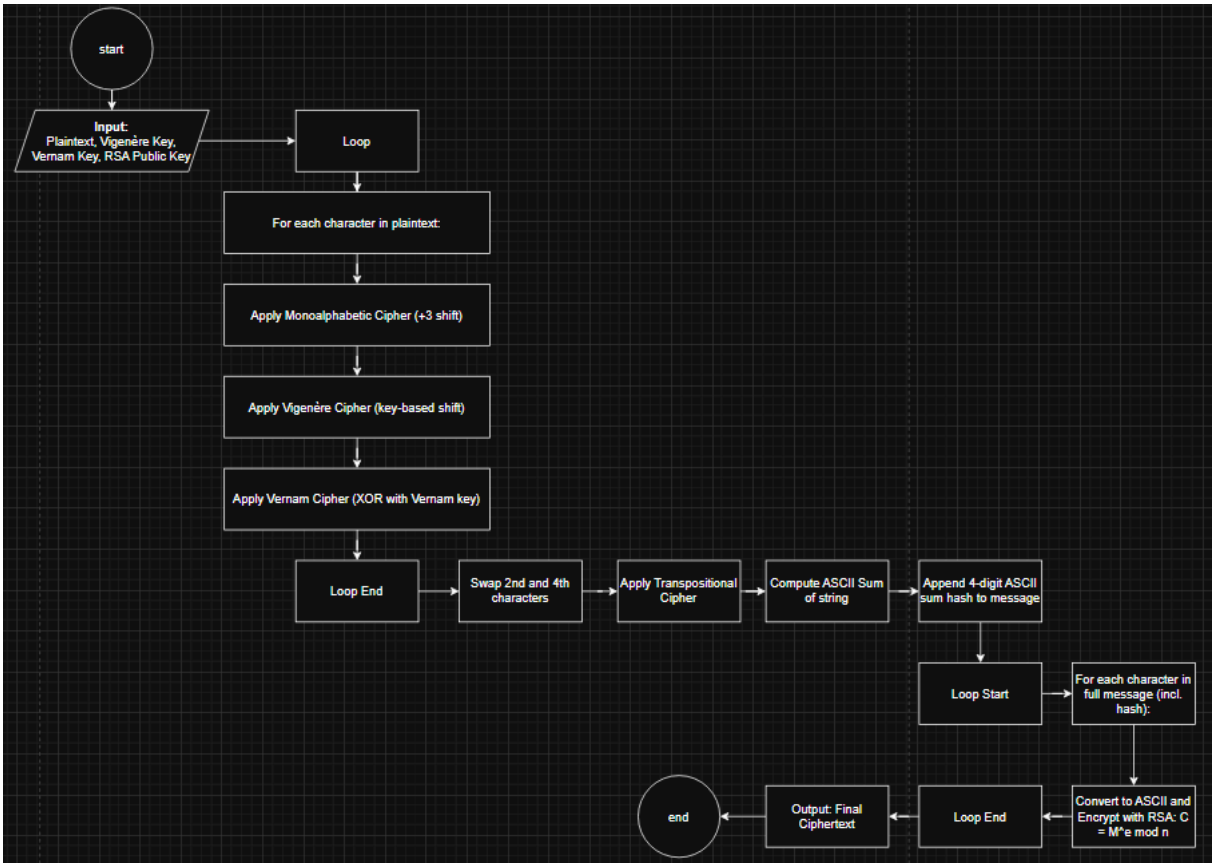
$$C = M \bmod n \quad C = M^e \bmod n \quad C = M \bmod n$$

where M is the message segment, e is the encryption exponent, and n is the modulus. This step ensures that the ciphertext can only be decrypted by the intended recipient who holds the private key.

9. **Ciphertext Output**

The fully encrypted message is presented as a sequence of RSA-encrypted integers or encoded ASCII values. This output may also be accompanied by a checksum or hash (e.g., SHA-256) for data integrity verification during transmission.

Encryption Flowchart:



Summary of Encryption Layers

Stage	Technique	Type	Primary Purpose
1	Monoalphabetic Cipher	Substitution	Initial confusion
2	Vigenère Cipher	Substitution	Polyalphabetic variability
3	Character Swapping	Permutation	Structural obfuscation
4	Vernam Cipher (XOR)	Symmetric	Binary-level randomness
5	Transpositional Cipher	Permutation	Block-based diffusion
6	RSA Encryption	Asymmetric	Secure delivery and confidentiality

Step-by-Step Decryption Process

1. Inputs Required:

- **Encrypted Message** (RSA-encrypted blocks)
- **Vigenère Key**
- **Vernam Key**
- **RSA Private Key** (d, n)

2. RSA Decryption

For each encrypted block:

- Apply RSA decryption:
$$M = C^{d \bmod n} \bmod n$$

Where:
 - C = Encrypted block
 - d, n = Private RSA key
 - M = Decrypted block (numeric)
- Convert M to its corresponding ASCII character.
- Reconstruct the full decrypted string.

3. Separate Hash and Message

- Extract the **last 4 characters** as the **message hash**.
- The **remaining characters** represent the actual encrypted message.

4. Hash Verification (ASCII Sum)

- Compute the **ASCII sum** of the message characters.
- Extract the **last 4 digits** of this sum.
- Compare this value to the extracted message hash:
 - If **mismatch**: Abort decryption and display error.
 - If **match**: Proceed to the next step.

5. Reverse Transpositional Cipher

- **Reverse the character swapping logic** used during encryption.
(e.g., if blocks were reversed every 4 characters, apply the same logic in

reverse)

6. Vernam Cipher Decryption (XOR)

- XOR each character with the **same Vernam key**.
- This reverses the Vernam encryption, restoring the intermediate message.

7. Vigenère Cipher Decryption

- Apply the Vigenère decryption formula for each character:
$$P_i = (C_i - K_i + 26) \bmod 26$$
 - C_i = Encrypted character
 - K_i = Key character
 - P_i = Plaintext character

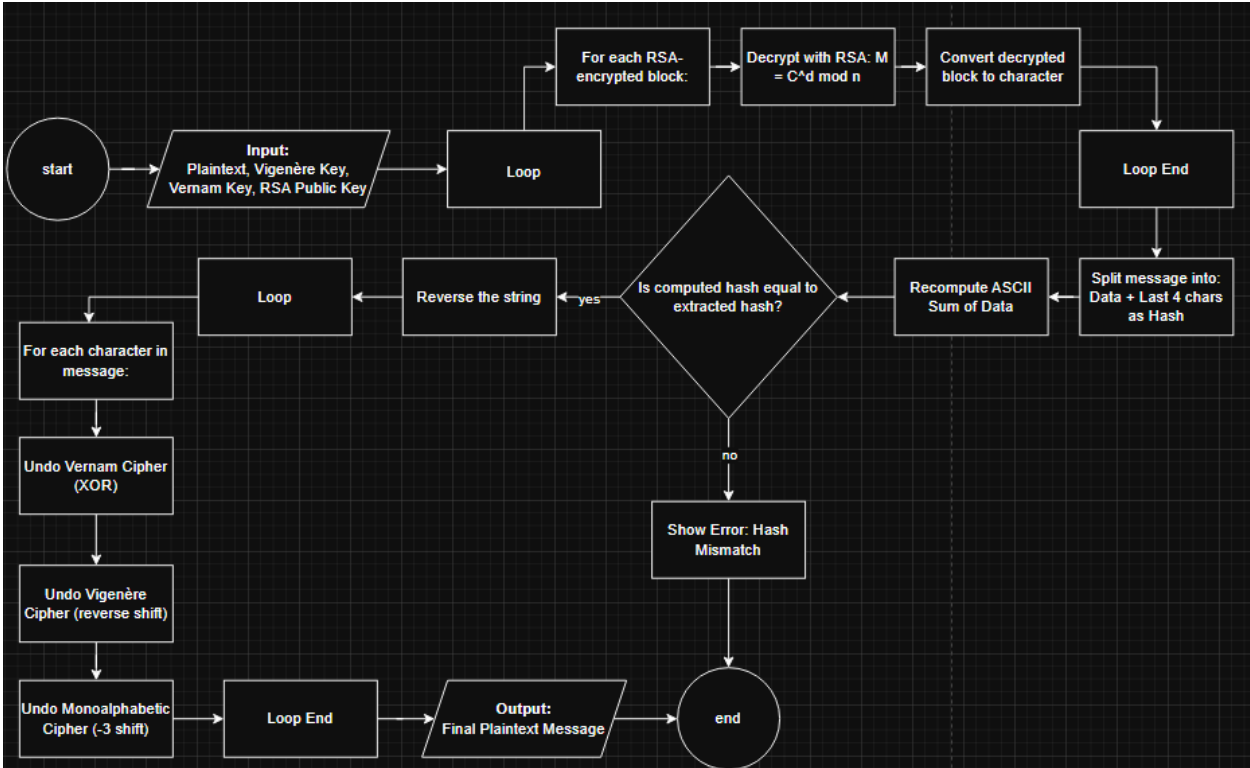
8. Monoalphabetic Cipher Decryption

- Reverse the fixed Caesar-style shift.
(e.g., if a +3 shift was applied during encryption, now apply a -3 shift)

9. Output

- The resulting string is the original **plaintext message**, now fully restored.

Decryption Flowchart:



Summary of Decryption Layers

Stage	Technique	Type	Primary Purpose
1	RSA Decryption	Asymmetric	Recover original encrypted message
2	Hash Verification (ASCII Sum)	Integrity Check	Ensure message authenticity
3	Transpositional Reversal	Permutation	Restore original block structure
4	Vernam Cipher (XOR)	Symmetric	Bitwise decryption
5	Vigenère Cipher Reversal	Substitution	Undo polyalphabetic shift
6	Monoalphabetic Reversal	Substitution	Final restoration of original text